

AI Virtual Mouse using Hand Tracking

Overview

This project implements an **AI Virtual Mouse** that allows users to control their computer's mouse cursor using hand gestures captured via a webcam. The system uses hand tracking to detect finger movements and gestures, enabling cursor movement and clicking functionality without physical hardware. The project leverages **OpenCV**, **MediaPipe**, and **Autopy** to process webcam input, detect hand landmarks, and control the mouse.

Features

- **Cursor Movement:** Move the mouse cursor by pointing with the index finger.
- **Clicking:** Perform a mouse click by pinching the index and middle fingers together.
- **Real-time FPS Display:** Shows the frame rate of the webcam feed for performance monitoring.
- **Smooth Cursor Movement:** Implements a smoothing mechanism to reduce jittery cursor movements.
- **Active Region:** Defines a rectangular area in the webcam feed for precise mouse control.

Requirements

To run this project, you need the following Python libraries:

- **OpenCV** (opencv-python): For capturing and processing webcam video.
- **MediaPipe** (mediapipe): For hand tracking and landmark detection.
- **Autopy** (autopy): For controlling the mouse cursor programmatically.
- **NumPy** (numpy): For numerical operations and coordinate mapping.

Install the dependencies using pip:

```
bash
```

```
Copy
```

```
pip install opencv-python mediapipe autopy numpy
```

Hardware Requirements

- A webcam (built-in or external) connected to your computer.
- A computer running Windows, macOS, or Linux (Autopy is cross-platform but may require additional setup on some systems).

Files

- **virtual_mouse.py:** The main script containing the virtual mouse implementation.
- **HandTrackingModule.py:** A custom module (handDetector class) for hand tracking using MediaPipe.

How It Works

1. **Hand Tracking:**

- The HandTrackingModule uses MediaPipe's hand tracking solution to detect hand landmarks in the webcam feed.
- It identifies key landmarks (e.g., fingertips) and determines which fingers are raised.

2. **Mouse Control:**

- **Movement:** When only the index finger is raised, its position is mapped from the webcam frame to the screen coordinates, moving the mouse cursor accordingly.
- **Clicking:** When both the index and middle fingers are raised and brought close together (pinch gesture), a mouse click is triggered.
- A smoothing factor is applied to reduce jitter in cursor movement.

3. **Active Region:**

- A rectangular region in the webcam feed is defined (with a border of 100 pixels by default) to limit the mouse control area, improving precision.

4. **FPS Display:**

- The frames per second (FPS) of the webcam feed is calculated and displayed on the video window for performance monitoring.

Usage

1. **Setup:**

- Ensure all dependencies are installed.
- Save both virtual_mouse.py and HandTrackingModule.py in the same directory.

2. **Run the Script:**

- Execute the main script:

```
bash
```

```
Copy
```

```
python virtual_mouse.py
```

- A window titled "AI Virtual Mouse" will open, showing the webcam feed with hand landmarks and a purple rectangle indicating the active mouse control area.

3. **Gestures:**

- **Move Cursor:** Raise only your index finger and move it within the active region to control the cursor.
- **Click:** Bring your index and middle fingers close together (pinch gesture) to perform a left mouse click.
- A green circle appears at the pinch point when a click is detected.

4. **Exit:**

- Press the q key to stop the program and close the webcam feed.

Code Structure

virtual_mouse.py

- Initializes the webcam and sets its resolution (640x480 by default).
- Uses the handDetector class from HandTrackingModule to detect hands and landmarks.
- Maps the index finger's position to screen coordinates for cursor movement.
- Detects pinch gestures (index and middle fingers) for clicking.
- Displays the webcam feed with landmarks, bounding box, and FPS.

HandTrackingModule.py

- Defines the handDetector class with methods for:
 - **findHands**: Detects hands in the image and draws landmarks (optional).
 - **findPosition**: Retrieves landmark coordinates and bounding box for a specified hand.
 - **fingersUp**: Determines which fingers are raised based on landmark positions.
 - **findDistance**: Calculates the distance between two landmarks (e.g., fingertips) and draws a line between them.
- Uses MediaPipe's hand tracking model for robust hand detection.

Configuration

You can adjust the following parameters in virtual_mouse.py:

- wCam, hCam: Webcam resolution (default: 640x480).
- frameR: Frame reduction (default: 100 pixels) to define the active mouse control area.
- smoothening: Smoothing factor for cursor movement (default: 7). Higher values make movement smoother but slower.
- length threshold in findDistance: Adjust the pinch distance for clicking (default: 40 pixels).

Troubleshooting

- **Webcam Not Working:**
 - Ensure your webcam is connected and functional.
 - Try changing cv2.VideoCapture(0) to cv2.VideoCapture(1) if the default camera doesn't work.
- **Autopy Issues:**
 - Autopy may require additional setup on some operating systems (e.g., Linux may need libxdo or similar).
 - Check Autopy's documentation for platform-specific requirements.
- **No Hand Detection:**
 - Ensure good lighting and keep your hand within the webcam's field of view.
 - Adjust detectionCon and trackCon in HandTrackingModule.py for better detection sensitivity.

- **Cursor Jitter:**
 - Increase the smoothening value for smoother cursor movement.
 - Reduce webcam resolution (wCam, hCam) for faster processing.

Limitations

- Requires good lighting for accurate hand tracking.
- May not work well with multiple hands in the frame (configured for single-hand tracking by default).
- Autopy's mouse control may have compatibility issues on some systems.
- The pinch gesture threshold (length < 40) may need tuning based on hand size or webcam resolution.

Future Improvements

- Add support for additional gestures (e.g., right-click, scroll).
- Implement multi-hand tracking for more complex interactions.
- Optimize performance for lower-end devices by reducing processing overhead.
- Add a GUI for adjusting parameters like smoothening or frameR in real-time.

License

This project is open-source and available under the [MIT License](#).

Acknowledgments

- Built using [OpenCV](#), [MediaPipe](<https://mediapipe.dev/>), and [Autopy](#).
- Inspired by hand tracking tutorials and virtual mouse projects in the computer vision community.

Detailed Explanation of the AI Virtual Mouse Programs

Overview

The **AI Virtual Mouse** project enables users to control their computer's mouse cursor using hand gestures captured via a webcam. The system detects hand landmarks, tracks finger positions, and translates these into mouse movements and clicks. The project uses:

- **OpenCV:** For capturing and processing webcam video.
- **MediaPipe:** For hand tracking and landmark detection.
- **Autopy:** For programmatically controlling the mouse cursor.
- **NumPy:** For numerical operations like coordinate mapping.

The main script (virtual_mouse.py) integrates hand tracking with mouse control, while HandTrackingModule.py provides a reusable class for hand detection and gesture recognition.

Program 1: virtual_mouse.py

Purpose

This script implements the core functionality of the AI Virtual Mouse. It:

- Captures webcam video.
- Uses the handDetector class from HandTrackingModule.py to detect and track hand landmarks.
- Maps the index finger's position to control the mouse cursor.
- Detects a pinch gesture (index and middle fingers close together) to trigger a mouse click.
- Displays the webcam feed with visual feedback (landmarks, bounding box, FPS).

Detailed Code Breakdown

1. Imports and Setup

python

Copy

```
import cv2
```

```
import numpy as np
```

```
import HandTrackingModule as htm
```

```
import time
```

```
import autopsy
```

- cv2: OpenCV for webcam capture and image processing.
- np: NumPy for numerical operations, such as mapping coordinates.
- HandTrackingModule as htm: Custom module for hand detection.
- time: For calculating frames per second (FPS).
- autopsy: For controlling the mouse cursor programmatically.

2. Configuration Parameters

python

Copy

```
wCam, hCam = 640, 480
```

```
frameR = 100 # Frame Reduction (defines the active region for mouse control)
```

```
smoothing = 7 # Smoothing factor for mouse movement
```

- wCam, hCam: Sets the webcam resolution to 640x480 pixels.
- frameR: Defines a border (100 pixels) around the webcam frame, creating a smaller active region for mouse control.
- smoothing: Controls the smoothing of mouse movement to reduce jitter.

3. Initialization

python

Copy

```
pTime = 0 # Previous time for calculating FPS
```

```
plocX, plocY = 0, 0 # Previous location of the mouse cursor
```

```
clocX, clocY = 0, 0 # Current location of the mouse cursor
```

```
cap = cv2.VideoCapture(0)
```

```
cap.set(3, wCam) # Set camera width
```

```
cap.set(4, hCam) # Set camera height
```

```
detector = htm.handDetector(maxHands=1)
```

```
wScr, hScr = autopy.screen.size()
```

- Initializes variables for FPS and cursor position tracking.
- Sets up the webcam with the specified resolution.
- Creates a handDetector instance for single-hand tracking.
- Retrieves the screen resolution for coordinate mapping.

4. Main Loop

The main loop processes each webcam frame as follows:

a. Capture and Preprocess Frame

python

Copy

```
success, img = cap.read()
```

```
if not success:
```

```
    print("Failed to grab frame from camera. Exiting...")
```

```
    break
```

```
img = cv2.flip(img, 1)
```

- Captures a frame and checks for success.
- Flips the image horizontally for a mirror effect.

b. Hand Detection

python

Copy

```
img = detector.findHands(img)
```

```
lmList, bbox = detector.findPosition(img, draw=True)
```

- Detects hands and draws landmarks.
- Retrieves landmark coordinates and bounding box.

c. Process Hand Landmarks

python

Copy

```
if len(lmList) != 0:
```

```
    x1, y1 = lmList[8][1], lmList[8][2] # Index finger tip
```

```
    x2, y2 = lmList[12][1], lmList[12][2] # Middle finger tip
```

```
    fingers = detector.fingersUp()
```

```
    cv2.rectangle(img, (frameR, frameR), (wCam - frameR, hCam - frameR), (255, 0, 255), 2)
```

- Extracts coordinates of index and middle finger tips.
- Determines which fingers are raised.
- Draws the active mouse control region.

d. Moving Mode

python

Copy

```
if fingers[1] == 1 and fingers[2] == 0:
```

```
    x3 = np.interp(x1, (frameR, wCam - frameR), (0, wScr))
```

```
    y3 = np.interp(y1, (frameR, hCam - frameR), (0, hScr))
```

```
    clocX = plocX + (x3 - plocX) / smoothening
```

```
    clocY = plocY + (y3 - plocY) / smoothening
```

```
    autopy.mouse.move(clocX, clocY)
```

```
    cv2.circle(img, (x1, y1), 15, (255, 0, 255), cv2.FILLED)
```

```
    plocX, plocY = clocX, clocY
```

- Maps the index finger's position to screen coordinates.
- Smooths cursor movement using a weighted average.

- Moves the cursor and draws a visual cue.

e. Clicking Mode

python

Copy

```
if fingers[1] == 1 and fingers[2] == 1:
```

```
    length, img, lineInfo = detector.findDistance(8, 12, img)
```

```
    if length < 40:
```

```
        cv2.circle(img, (lineInfo[4], lineInfo[5]), 15, (0, 255, 0), cv2.FILLED)
```

```
        autopy.mouse.click()
```

- Detects a pinch gesture and triggers a mouse click.
- Draws a green circle at the pinch point.

f. FPS Display

python

Copy

```
cTime = time.time()
```

```
fps = 1 / (cTime - pTime)
```

```
pTime = cTime
```

```
cv2.putText(img, f'FPS: {int(fps)}', (20, 50), cv2.FONT_HERSHEY_PLAIN, 3, (255, 0, 0), 3)
```

- Calculates and displays FPS.

g. Display and Exit

python

Copy

```
cv2.imshow("AI Virtual Mouse", img)
```

```
if cv2.waitKey(1) & 0xFF == ord('q'):
```

```
    break
```

```
cap.release()
```

```
cv2.destroyAllWindows()
```

- Displays the processed frame and exits on q.

Program 2: HandTrackingModule.py

Purpose

Defines the handDetector class for hand tracking using MediaPipe, providing methods to detect hands, retrieve landmarks, identify raised fingers, and calculate distances.

Detailed Code Breakdown

1. Imports

python

Copy

```
import cv2
```

```
import mediapipe as mp
```

```
import math
```

2. Class Definition and Initialization

python

Copy

```
class handDetector():
```

```
    def __init__(self, mode=False, maxHands=2, detectionCon=0.5, trackCon=0.5):
```

```
        self.mode = mode
```

```
        self.maxHands = maxHands
```

```
        self.detectionCon = detectionCon
```

```
        self.trackCon = trackCon
```

```
        self.mpHands = mp.solutions.hands
```

```
        self.hands = self.mpHands.Hands(
```

```
            static_image_mode=self.mode,
```

```
            max_num_hands=self.maxHands,
```

```
            min_detection_confidence=self.detectionCon,
```

```
            min_tracking_confidence=self.trackCon
```

```
        )
```

```
        self.mpDraw = mp.solutions.drawing_utils
```

```
        self.tipIds = [4, 8, 12, 16, 20]
```

- Configures MediaPipe for hand tracking with customizable parameters.

3. findHands Method

python

Copy

```

def findHands(self, img, draw=True):

    imgRGB = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    self.results = self.hands.process(imgRGB)

    if self.results.multi_hand_landmarks:

        for handLms in self.results.multi_hand_landmarks:

            if draw:

                self.mpDraw.draw_landmarks(img, handLms, self.mpHands.HAND_CONNECTIONS)

    return img

```

- Detects hands and draws landmarks if specified.

4. findPosition Method

python

Copy

```

def findPosition(self, img, handNo=0, draw=True):

    self.lmList = []

    self.bbox = (0, 0, 0, 0)

    if self.results.multi_hand_landmarks:

        myHand = self.results.multi_hand_landmarks[handNo]

        x_coords = []

        y_coords = []

        for id, lm in enumerate(myHand.landmark):

            h, w, c = img.shape

            cx, cy = int(lm.x * w), int(lm.y * h)

            self.lmList.append([id, cx, cy])

            x_coords.append(cx)

            y_coords.append(cy)

        x_min, y_min = min(x_coords), min(y_coords)

        x_max, y_max = max(x_coords), max(y_coords)

        self.bbox = (x_min, y_min, x_max - x_min, y_max - y_min)

        if draw:

            bx, by, bw, bh = self.bbox

```

```
cv2.rectangle(img, (bx, by), (bx + bw, by + bh), (255, 0, 255), 2)
```

```
return self.lmList, self.bbox
```

- Retrieves landmark coordinates and bounding box.

5. fingersUp Method

python

Copy

```
def fingersUp(self):  
    fingers = []  
  
    if self.lmList:  
        if self.lmList[self.tipIds[0]][1] > self.lmList[self.tipIds[0] - 1][1]:  
            fingers.append(1)  
        else:  
            fingers.append(0)  
  
        for id in range(1, 5):  
            if self.lmList[self.tipIds[id]][2] < self.lmList[self.tipIds[id] - 2][2]:  
                fingers.append(1)  
            else:  
                fingers.append(0)  
  
    return fingers
```

- Identifies raised fingers based on landmark positions.

6. findDistance Method

python

Copy

```
def findDistance(self, p1, p2, img, draw=True, r=15, t=3):  
    if len(self.lmList) != 0:  
        x1, y1 = self.lmList[p1][1:]  
        x2, y2 = self.lmList[p2][1:]  
        cx, cy = (x1 + x2) // 2, (y1 + y2) // 2  
  
        if draw:  
            cv2.line(img, (x1, y1), (x2, y2), (255, 0, 255), t)
```

```

cv2.circle(img, (x1, y1), r, (255, 0, 255), cv2.FILLED)

cv2.circle(img, (x2, y2), r, (255, 0, 255), cv2.FILLED)

cv2.circle(img, (cx, cy), r, (0, 0, 255), cv2.FILLED)

length = math.hypot(x2 - x1, y2 - y1)

return length, img, [x1, y1, x2, y2, cx, cy]

return 0, img, [0,0,0,0,0,0]

```

- Calculates distance between two landmarks and provides visual feedback.

7. Main Function (for Testing)

python

Copy

```

def main():

    cap = cv2.VideoCapture(0)

    detector = handDetector()

    pTime = 0

    while True:

        success, img = cap.read()

        img = detector.findHands(img)

        lmList, bbox = detector.findPosition(img)

        if len(lmList) != 0:

            fingers = detector.fingersUp()

            length, img, lineInfo = detector.findDistance(8, 12, img)

        cTime = time.time()

        fps = 1 / (cTime - pTime)

        pTime = cTime

        cv2.putText(img, str(int(fps)), (10, 70), cv2.FONT_HERSHEY_PLAIN, 3, (255, 0, 255), 3)

        cv2.imshow("Image", img)

        if cv2.waitKey(1) & 0xFF == ord('q'):

            break

    cap.release()

    cv2.destroyAllWindows()

```

- Tests the handDetector class independently.

How the Programs Work Together

1. **Initialization:** virtual_mouse.py creates a handDetector instance and sets up the webcam and screen resolution.
2. **Hand Tracking:** Calls findHands and findPosition to detect hands and retrieve landmarks.
3. **Gesture Recognition:** Uses fingersUp and findDistance to detect moving and clicking gestures.
4. **Mouse Control:** Maps hand positions to cursor movements and triggers clicks.
5. **Visual Feedback:** Displays landmarks, bounding box, active region, and FPS.

Key Concepts

- **Hand Landmarks:** MediaPipe provides 21 landmarks per hand (IDs 0–20).
- **Coordinate Mapping:** Scales webcam coordinates to screen coordinates.
- **Smoothing:** Reduces cursor jitter using a weighted average.
- **Gesture Detection:** Uses finger states and distances for control.
- **Active Region:** Confines mouse control to a smaller area for precision.

Potential Improvements

- Support multi-hand gestures.
- Add right-click or scroll functionality.
- Dynamically adjust pinch threshold.
- Optimize for lower-end devices.
- Enhance cross-platform compatibility.

Limitations

- Requires good lighting for hand tracking.
- Assumes right-hand orientation for thumb detection.
- Autopy may have compatibility issues.
- Limited to single-hand tracking.

Conclusion

The AI Virtual Mouse combines computer vision and gesture recognition for intuitive mouse control. The virtual_mouse.py script handles application logic, while HandTrackingModule.py provides robust hand tracking. The system is extensible and demonstrates the power of MediaPipe and Autopy.